Senior *Rust Trading* Systems Engineer

Location: Remote / Poland (International Team)
Employment Type: Full-time / Part-time / Contract
Reports To: Head of Engineering / CTO
Level: Senior Engineer
Role Type: Hands-on Development, *Trading* Infrastructure & Distributed Systems

About the Company

We are planning to establish a new software development company in Poland, focused on building modern fintech, payment infrastructure, and *blockchain*-based financial applications.

Our planned products cover several areas:

- **Fintech & Payments:** Digital payments, transaction processing, financial integrations, and settlement infrastructure.
- **Real Estate Technology:** Property marketplaces with integrated financial services and digital payments.
- **Gaming & Sports Betting:** Real-time financial transactions, market data, betting infrastructure, and payment settlement.
- ***Web3* Social Finance:** *Blockchain*-connected applications, digital assets, decentralized *trading* integrations, and financial services.

Our goal is to combine traditional financial technology with *blockchain* infrastructure to develop scalable, secure, and reliable financial applications.

We are building an international engineering team and looking for experienced engineers who can contribute directly to system architecture and development from an early stage.

About the Role

We are looking for an experienced **Senior *Rust Trading* Systems Engineer** to design, build, and optimize high-performance backend systems supporting financial *trading*, market data, and *blockchain*-connected transactions.

This is a hands-on engineering position focused on ***Rust* development, low-latency systems, *trading* infrastructure, distributed processing, and financial transaction reliability**.

You will work closely with backend engineers, *blockchain* developers, infrastructure engineers, and product teams to build services that process market data, manage *trading* workflows, and interact with traditional and decentralized financial systems.

Your responsibilities will include developing market data ingestion services, *trading* execution infrastructure, transaction-processing systems, and real-time backend services.

You will also contribute to system architecture, performance optimization, and production reliability.

We are looking for someone who understands both software engineering and financial transaction workflows, with the ability to write efficient *Rust* code and solve complex systems problems.

This is not a purely architectural or management role. You will actively develop production software, review code, investigate performance bottlenecks, and take ownership of important backend components.

What You'll Do

1. *Rust* Backend & Systems Engineering

- Design, develop, and maintain high-performance backend services using *Rust*.
- Build concurrent and asynchronous services using Tokio and related *Rust* libraries.
- Develop reliable APIs and internal service interfaces.
- Optimize memory usage, CPU performance, throughput, and processing latency.
- Implement efficient task scheduling, buffering, batching, and event-processing mechanisms.
- Develop reusable components and libraries for *trading* and financial infrastructure.
- Investigate performance bottlenecks, race conditions, and resource contention.
- Apply *Rust* ownership, borrowing, concurrency, and error-handling principles.
- Write clean, maintainable, well-tested production code.
- Participate in technical architecture discussions and code reviews.

2. *Trading* Systems & Execution Infrastructure

- Develop backend infrastructure supporting *trading* operations and execution workflows.
- Build services for receiving, validating, routing, and tracking orders.
- Implement reliable order-state management and execution-status synchronization.
- Design integrations with centralized exchanges, decentralized exchanges, and *trading* APIs.
- Develop order-routing and execution components where required.
- Handle order submission, cancellation, partial fills, execution errors, and reconciliation.
- Build systems that can recover safely from network interruptions and service failures.

- Implement idempotency, retry mechanisms, and duplicate-event protection.
- Optimize execution pipelines for latency, reliability, and throughput.
- Collaborate with product and financial engineering teams to ensure correct *trading* behavior.

3. Market Data Infrastructure

- Design and develop high-throughput market data ingestion services.
- Integrate exchange market data feeds using WebSocket, REST, FIX, or other relevant protocols.
- Process real-time trade events, price updates, and order book data.
- Implement order book reconstruction and incremental update processing.
- Normalize market data from multiple providers into consistent internal formats.
- Handle out-of-order messages, duplicate events, missing updates, and reconnect scenarios.
- Develop streaming pipelines for downstream *trading* and analytics services.
- Optimize ingestion services for low latency and efficient resource utilization.
- Build historical market data processing and replay capabilities.
- Monitor feed quality, processing lag, and data consistency.

4. *Blockchain* & *DeFi Trading* Infrastructure

- Develop infrastructure supporting trading across EVM-compatible networks and other blockchain ecosystems.
- Integrate decentralized exchange protocols and liquidity aggregators.
- Build backend services for on-chain trade execution and transaction monitoring.
- Develop integrations with blockchain RPC services and transaction submission APIs.
- Handle transaction confirmation, failures, replacement transactions, and chain reorganizations.
- Support trading workflows involving swaps, token transfers, and decentralized liquidity pools.
- Integrate wallet and transaction-signing services through secure interfaces.
- Develop reliable deposit and withdrawal monitoring services.
- Implement blockchain event processing and transaction reconciliation.
- Collaborate with smart contract engineers when integrating on-chain functionality.

The primary responsibility is building reliable off-chain trading infrastructure and blockchain integrations, rather than developing core blockchain consensus software.

5. Financial Transaction Processing

- Build backend systems supporting digital asset transactions and payment-related workflows.
- Develop reliable transaction state management and event processing.

- Implement services for deposits, withdrawals, transfers, and settlement tracking.
- Design reconciliation processes between external networks and internal financial records.
- Handle incomplete transactions, retries, duplicate requests, and unexpected failures.
- Develop accurate financial transaction history and reporting services.
- Support integration with traditional payment systems and cryptocurrency infrastructure.
- Implement appropriate validation and authorization mechanisms.
- Collaborate with engineering teams to maintain financial data consistency.
- Ensure transaction-processing systems are observable, testable, and recoverable.

6. Distributed Systems & Data Processing

- Design and maintain event-driven and distributed backend services.
- Build reliable communication between trading engines, data services, and financial applications.
- Work with Kafka, Redis, RabbitMQ, NATS, or comparable messaging technologies.
- Develop scalable consumers and producers for high-volume event streams.
- Implement reliable message processing and recovery mechanisms.
- Optimize data pipelines involving PostgreSQL and other databases.
- Design efficient database schemas and indexing strategies for financial data.
- Implement caching, batching, and backpressure management.
- Support high-throughput transaction and market data processing.
- Investigate consistency and performance issues across distributed components.

7. Performance Optimization & Reliability

- Benchmark trading and market data services under realistic workloads.
- Measure and improve latency, throughput, CPU utilization, and memory consumption.
- Profile Rust applications to identify inefficient processing paths.
- Optimize data structures, serialization, network communication, and database access.
- Implement appropriate timeout, retry, and circuit-breaker mechanisms.
- Develop recovery strategies for service outages and interrupted processing.
- Build monitoring for transaction failures, market data delays, and service health.
- Use structured logging, metrics, and distributed tracing.
- Work with Prometheus, Grafana, OpenTelemetry, or similar technologies.
- Participate in production incident investigation and long-term reliability improvements.

8. Testing & Engineering Quality

- Write unit, integration, and end-to-end tests for critical systems.
- Develop tests for order processing, market data parsing, and transaction workflows.
- Test systems against duplicate messages, missing events, timeouts, and unexpected failures.
- Build load-testing and performance-benchmarking tools.

- Use historical market data or simulated environments to validate trading behavior.
- Test blockchain integrations against appropriate development networks and local environments.
- Contribute to CI/CD pipelines and automated regression testing.
- Review code for correctness, performance, maintainability, and security.
- Document important technical decisions and system behavior.
- Support engineering standards across the backend development team.

9. Technical Ownership & Collaboration

- Take ownership of important Rust services from architecture through production deployment.
- Collaborate with backend, blockchain, frontend, QA, and infrastructure engineers.
- Participate in architecture reviews and technical planning.
- Help define service boundaries and integration requirements.
- Mentor engineers through code reviews and technical discussions.
- Identify technical risks and recommend practical solutions.
- Contribute to development estimates and implementation planning.
- Communicate technical trade-offs clearly to engineering and product teams.
- Remain actively involved in hands-on development.

Required Skills & Experience

- 5+ years of professional software engineering experience.
- 2+ years of hands-on Rust development in production systems.
- Strong understanding of Rust ownership, borrowing, lifetimes, traits, and error handling.
- Experience with asynchronous Rust programming using Tokio or comparable frameworks.
- Strong knowledge of concurrency, networking, and distributed systems.
- Experience developing high-performance backend services.
- Practical experience with trading systems, market data processing, financial infrastructure, or cryptocurrency transaction systems.
- Understanding of order processing, execution workflows, or financial transaction lifecycles.
- Experience working with WebSocket, REST, gRPC, or similar communication protocols.
- Familiarity with real-time event processing and message-driven architectures.
- Experience with PostgreSQL, Redis, or other production databases.
- Understanding of database transactions, consistency, and performance optimization.
- Experience designing reliable services that handle failures and retries.
- Knowledge of profiling, benchmarking, and performance testing.
- Experience with Linux, Docker, Git, and CI/CD workflows.
- Strong debugging and problem-solving skills.
- Ability to independently design, implement, test, and maintain production services.
- Comfortable working in a remote, internationally distributed engineering team.

- Professional English communication skills.

Nice to Have (Differentiators)

- Experience with financial market data protocols such as ITCH, FIX, or FAST.
- Experience building market data feed handlers or order book reconstruction engines.
- Knowledge of exchange matching engines, order management systems, or execution platforms.
- Experience developing low-latency trading or market data infrastructure.
- Familiarity with cryptocurrency exchanges and decentralized trading protocols.
- Experience integrating Uniswap, Raydium, or other decentralized exchanges.
- Knowledge of Ethereum, Solana, and EVM-compatible blockchain networks.
- Experience with on-chain transaction monitoring and confirmation tracking.
- Familiarity with blockchain reorganizations and transaction reconciliation.
- Experience with wallet infrastructure, cryptocurrency deposits, and withdrawals.
- Understanding of DeFi liquidity pools, swaps, and execution routing.
- Experience with Kafka, NATS, RabbitMQ, or other streaming platforms.
- Experience with PostgreSQL query optimization and high-volume financial datasets.
- Knowledge of Java, Go, C++, or TypeScript.
- Experience migrating performance-sensitive services from another language to Rust.
- Familiarity with Tauri or Rust-based desktop applications.
- Experience with OpenTelemetry, Prometheus, Grafana, or distributed tracing.
- Understanding of quantitative trading concepts and financial market microstructure.
- Experience in fintech, brokerage platforms, crypto exchanges, or digital asset infrastructure.
- Open-source contributions to Rust, financial infrastructure, or blockchain projects.

Senior-Level Expectations

As a Senior Rust Trading Systems Engineer, you will be expected to:

- Independently design and develop performance-critical backend services.
- Take ownership of complex Rust components and their production reliability.
- Make practical technical decisions involving performance, security, and maintainability.
- Understand financial transaction lifecycles and trading system requirements.
- Build systems that behave correctly under heavy load and unexpected failures.
- Identify performance bottlenecks and implement measurable improvements.
- Design reliable integrations with trading venues and blockchain infrastructure.
- Maintain high engineering standards through testing, code review, and documentation.
- Mentor other engineers and contribute to technical discussions.
- Collaborate closely with product and engineering leadership.
- Remain actively involved in coding, debugging, and architecture.

This is a senior individual contributor role. Formal people management is not required, but technical ownership and collaboration are important.

Example Problems You'll Work On

1. High-Throughput Market Data Processing

Build a Rust service that processes a continuous stream of market data updates.

The system should handle high message volumes, maintain consistent state, and recover safely from interruptions or missing updates.

2. Real-Time Order Book Reconstruction

Develop an order book service that processes snapshots and incremental updates.

Ensure bids, asks, and price levels remain correct when messages arrive out of order or a connection is interrupted.

3. Multichain Trading Execution

Build a backend service that routes trading requests to appropriate decentralized exchanges.

Support transaction submission, execution status tracking, error handling, and reconciliation.

4. Cryptocurrency Deposit Monitoring

Develop a service that monitors deposits across supported blockchain networks.

Track confirmations, handle reorganizations, and synchronize transaction states with internal systems.

5. Low-Latency Rust Optimization

Investigate a Rust service experiencing increased latency under load.

Use profiling and benchmarking to identify bottlenecks, optimize resource usage, and demonstrate measurable improvements.

6. Distributed Trade Processing

Design a queue-based trading execution system.

Ensure orders are processed reliably even when services restart, messages are delivered more than once, or external trading venues become temporarily unavailable.

7. Financial Data Reconciliation

Develop a reconciliation service that compares internal transaction records with external exchange or blockchain data.

Identify missing, duplicated, or inconsistent records and support safe recovery workflows.

8. Trading System Observability

Implement metrics, logs, and distributed tracing for trading and market data services.

Provide visibility into execution latency, failed transactions, processing delays, and system reliability.

Initial Responsibilities

During the first stage of development, the Senior Rust Trading Systems Engineer will focus on the following priorities.

System Architecture: Review the proposed financial and trading infrastructure, identify components requiring Rust, and help establish appropriate service boundaries and technical standards.

Core Engineering: Build selected backend services for market data ingestion, transaction processing, trading integrations, or financial event processing.

Reliability & Performance: Establish meaningful performance benchmarks, automated testing, monitoring, and recovery mechanisms for critical services.

Engineering Collaboration: Work with other engineers to integrate Rust services with the wider application infrastructure and contribute to technical decisions.

The exact priorities will depend on the initial product roadmap.

Why This Role Matters

Our planned fintech and Web3 applications will depend on reliable financial data processing, secure transactions, and high-performance backend infrastructure.

Trading platforms and blockchain-connected financial systems must process events accurately, respond quickly, and recover safely from unexpected failures.

Even small errors in transaction state management, market data processing, or execution workflows can create significant financial and operational risks.

The Senior Rust Trading Systems Engineer will help build the infrastructure that supports these requirements.

This role offers the opportunity to work on challenging engineering problems involving low-latency systems, financial transactions, real-time data processing, and blockchain integrations.

You will contribute directly to technical architecture, develop production Rust services, and help establish reliable engineering practices as our international team grows.

We are looking for an engineer who combines **strong Rust programming skills, practical trading or financial systems experience, distributed systems knowledge, and a hands-on approach to building reliable, high-performance software**.